perplexity

Сделай анимацию изменения задержки во времени

Ниже — обновлённый скрипт `model_animation.py`. Он создаёт **анимацию изменения задержки помощи во времени** и сохраняет её в GIF. Перед запуском обязательно проверяет наличие `numpy`, `matplotlib` и `pillow`; если чего-то нет — предлагает установить.

Matplotlib поддерживает сохранение анимаций через `PillowWriter` в формат `.gif`; `FuncAnimation` обновляет кадры через функцию `update`. [1] [2] [3]

Сохрани код как `model_animation.py` на рабочий стол и запусти:

```
python model_animation.py
```

```
# -*- coding: utf-8 -*-
"""
Анимация универсальной модели ответственности:

V(T) = B(T) - R(T) + ∫ D(t) * S(t) dt

D(t) = exp(-lambda * t)

Анимация показывает:
- как уменьшается эффективность помощи D(t) при задержке;
- как падает итоговая ценность V(T);
- сравнение человека, финансового кризиса и нейросети.
"""

import importlib.util
import subprocess
import sys

REQUIRED = ["numpy", "matplotlib", "pillow"]


def check_libraries():
    missing = []

    for package in REQUIRED:
        spec = importlib.util.find_spec(package)
        if spec is None:
            missing.append(package)
            print(f"[НЕТ] {package} не установлен.")
        else:
            print(f"[ОК] {package} установлен.")
```

```
    if missing:
        answer = input(
            f"\nДля работы нужны: {', '.join(missing)}.\n"
            "Установить их сейчас через pip? [y/n]: "
        ).strip().lower()

        if answer in ("y", "yes", "д", "да"):
            for package in missing:
                print(f"Устанавливаю {package}...")
                subprocess.check_call(
                    [sys.executable, "-m", "pip", "install", package]
                )
        else:
            print("\nУстановка отменена. Скрипт завершён.")
            sys.exit(1)

    for package in REQUIRED:
        if importlib.util.find_spec(package) is None:
            print(f"Ошибка: {package} не удалось установить.")
            sys.exit(1)

    print("\nВсе необходимые библиотеки готовы.\n")


check_libraries()

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.animation import FuncAnimation, PillowWriter


# ----------------------------
# Модель
# ----------------------------

def delay_coefficient(t, lam=0.15, critical_time=1.0):
    t = np.asarray(t, dtype=float)
    return np.where(
        t <= critical_time,
        1.0,
        np.exp(-lam * (t - critical_time))
    )


def model_value(B, R, S, lam, critical_time=1.0, T=20.0, steps=1000):
    t = np.linspace(0, T, steps)
    dt = t[^1] - t[^0]
    D = delay_coefficient(t, lam, critical_time)
    support_integral = np.sum(D * S * dt)
    return B - R + support_integral


# ----------------------------
# Параметры анимации
# ----------------------------
```

```
T_MAX = 20.0
FRAMES = 120
LAMBDA = 0.18
CRITICAL_TIME = 1.0

t_values = np.linspace(0, T_MAX, 500)

B = 100.0
R = 80.0
S = 60.0

fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 8))

# Верхний график: коэффициент задержки
line_D, = ax1.plot(
    [], [],
    color="#c0392b",
    linewidth=3,
    label="Эффективность помощи D(t)"
)

ax1.set_xlim(0, T_MAX)
ax1.set_ylim(0, 1.1)
ax1.set_xlabel("Время задержки t")
ax1.set_ylabel("D(t)")
ax1.set_title("Задержка помощи: чем позже реакция, тем ниже эффективность")
ax1.grid(True, alpha=0.3)
ax1.legend(loc="upper right")

# Нижний график: V(T)
line_V, = ax2.plot(
    [], [],
    color="#2e86de",
    linewidth=3,
    label="Итоговая ценность V(T)"
)

ax2.set_xlim(0, T_MAX)
ax2.set_ylim(-20, 100)
ax2.set_xlabel("Время задержки t")
ax2.set_ylabel("V(T)")
ax2.set_title("Итоговая ценность системы V(T)")
ax2.grid(True, alpha=0.3)
ax2.legend(loc="upper right")

# Текст с текущими значениями
text = ax2.text(
    0.02, 0.90,
    "",
    transform=ax2.transAxes,
    fontsize=11,
    family="monospace",
    bbox=dict(facecolor="white", alpha=0.8)
)
```

```
def update(frame):
    current_t = T_MAX * frame / (FRAMES - 1)

    D_current = delay_coefficient(
        np.array([current_t]),
        lam=LAMBDA,
        critical_time=CRITICAL_TIME
    )[^0]

    V_current = model_value(
        B=B,
        R=R,
        S=S,
        lam=LAMBDA,
        critical_time=CRITICAL_TIME,
        T=current_t
    )

    t_plot = t_values[t_values <= current_t]
    D_plot = delay_coefficient(
        t_plot,
        lam=LAMBDA,
        critical_time=CRITICAL_TIME
    )

    V_plot = []
    for time_point in t_plot:
        value = model_value(
            B=B,
            R=R,
            S=S,
            lam=LAMBDA,
            critical_time=CRITICAL_TIME,
            T=time_point
        )
        V_plot.append(value)

    line_D.set_data(t_plot, D_plot)
    line_V.set_data(t_plot, V_plot)

    text.set_text(
        f"t = {current_t:.2f}\n"
        f"D(t) = {D_current:.3f}\n"
        f"V(T) = {V_current:.2f}"
    )

    return line_D, line_V, text


ani = FuncAnimation(
    fig,
    update,
    frames=FRAMES,
    interval=60,
    blit=False,
```

```
    repeat=True
)

# Сохранение анимации в GIF
ani.save(
    "model_delay_animation.gif",
    writer=PillowWriter(fps=15),
    dpi=120
)

print("Готово. Создан файл: model_delay_animation.gif")
```

Что показывает анимация

- **Верхний график**: как коэффициент задержки $D(t)$ падает со временем.
- **Нижний график**: как падает итоговая ценность $V(T)$.
- **Текст**: текущие значения t, $D(t)$, $V(T)$.
- **Скорость падения**: чем больше λ, тем быстрее теряется эффект помощи.

Ключевая идея

$$\boxed{D(t) = e^{-\lambda t}}$$

И:

$$\boxed{V(T) = B(T) - R(T) + \int_0^T D(t)S(t)dt}$$

Анимация наглядно показывает:

- при $t = 0$ помощь максимальна;
- при росте задержки $D(t)$ падает;
- при большой задержке даже большая сумма помощи даёт малый эффект;
- при очень долгом промедлении $V(T)$ может стать отрицательной.

Как запустить

1. Сохрани код как `model_animation.py` на рабочем столе.
2. Открой терминал в папке рабочего стола.
3. Выполни:

```
python model_animation.py
```

После завершения появится файл:

```
model_delay_animation.gif
```

Почему GIF

GIF выбран потому, что его можно сохранить и открыть без дополнительных кодеков. Matplotlib поддерживает сохранение анимаций в `.gif` через `PillowWriter`; для `.mp4` обычно требуется установленный `ffmpeg`.[2] [4] [1]

Финальный смысл

Анимация показывает главную идею модели:

> Помощь, пришедшая слишком поздно, уже не полностью искупает вред.

Именно это делает модель живой: она показывает не статичную формулу, а **процесс утраты ценности помощи во времени**.[3] [1] [2]

⁂

1. https://matplotlib.org/3.7.5/tutorials/introductory/animation_tutorial.html
2. https://matplotlib.org/stable/users/explain/animations/animations.html
3. https://foundations.projectpythia.org/core/matplotlib/histograms-piecharts-animation/
4. https://matplotlib.org/3.9.0/users/explain/animations/animations.html
5. https://matplotlib.org/stable/api/_as_gen/matplotlib.animation.FFMpegWriter.html
6. https://matplotlib.org/2.1.2/gallery/animation/basic_example_writer_sgskip.html
7. https://matplotlib.org/stable/gallery/animation/simple_scatter.html
8. https://fr.matplotlib.net/stable/api/_as_gen/matplotlib.animation.FFMpegWriter.html
9. https://www.resus.org.uk/professional-library/2025-resuscitation-guidelines/ethics-guidelines
10. https://matplotlib.org/stable/api/animation_api.html
11. https://matplotlib.org/3.1.1/_modules/matplotlib/animation.html
12. https://matplotlib.org/2.0.2/api/_as_gen/matplotlib.animation.Animation.save.html
13. https://matplotlib.org/3.8.2/api/animation_api.html